

Object-Oriented Programming

Overloading and Overriding

Team PBO

What will we learn

- Overloading
 - Overriding
- 

Overloading

- Overloading is a state where there are methods with the same name, but different signatures.
 - *A signature is the arrangement of parameters in a method declaration.*
- 

Overloading

- Overloading can occur within one class or in a parent class and subclass.
 - Characteristics of overloading:
 - The method name is the same.
 - The parameter list (signature) is different.
 - The return type can be the same or different.
- 

Example of Overloading

```
public class Manusia
{
    public void setBiodata(String nama)
    {
        System.out.println("Method setBiodata yg pertama dipanggil");
    }

    public void setBiodata(String nama, int umur)
    {
        System.out.println("Method setBiodata yg kedua dipanggil");
    }

    public void setBiodata(String nama, int umur, String alamat)
    {
        System.out.println("Method setBiodata yg ketiga dipanggil");
    }
}
```

The method name is the same, but its signature or arrangement of parameters is different.

- In the class Human above, there are three setBiodata() methods, but with different signatures.
- Observe the following TestHuman:

Example of Overloading

- If the class Human above is tested with the following class TestHuman:

```
public class TestManusia
{
    public static void main(String[] args)
    {
        Manusia man = new Manusia();
        man.setBiodata("Joko");
        man.setBiodata("Joko", 20, "Jakarta");
    }
}
```

Then the result will be:

```
D:\Java\Polimorfisme>java TestManusia
Method setBiodata yg pertama dipanggil
Method setBiodata yg ketiga dipanggil
```

Constructor Overloading

- Constructors can also be overloaded.

Example of Constructor Overloading

```
public class Kucing
{
    public Kucing()
    {
        System.out.println("Konstruktor yg pertama dipanggil");
    }

    public Kucing(String jenis)
    {
        System.out.println("Konstruktor yg kedua dipanggil");
    }
}
```

- In the class Cat above, there are two constructors, but with different signatures.
- Observe the following TestCat:

Example of Constructor Overloading

- If the class Cat above is tested with TestCat as follows:

```
public class TestKucing
{
    public static void main(String[] args)
    {
        Kucing jerry = new Kucing();
        Kucing garfield = new Kucing("Persian Mix");
    }
}
```

- Then the result will be.

```
D:\Java\Polimorfisme>java TestKucing
Konstruktor yg pertama dipanggil
Konstruktor yg kedua dipanggil
```

When to Use Overloading?

- To simplify code and maintain consistency in method naming
 - Because sometimes we need to create several methods that serve the same purpose, but have different parameters
- 

Real World Scenario of Overloading Usage

- For example, a motorcycle workshop can repair all types of motorcycles. However, the repair process varies depending on the type of motorcycle being handled.

```
public class BengkelMotor
{
    public void servis(MotorMatik mtk)
    {
        // lakukan proses perbaikan yang spesifik untuk motor matik
    }

    public void servis(MotorSport sprt)
    {
        // lakukan proses perbaikan yang spesifik untuk motor sport
    }
}
```

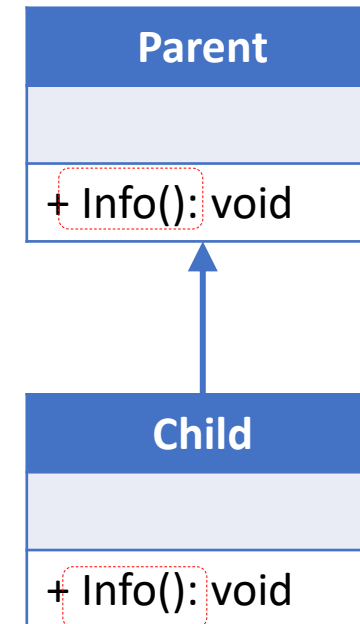
Overriding

- Overriding occurs when there is a method in the subclass with the same name and signature as its parent class.
 - Characteristics of overriding:
 - Occurs in subclasses/derived classes
 - The method name is the same.
 - The signature (arrangement of parameters) is the same.
 - The return type is the same.
- 

Overriding

```
class Parent
{
    public void Info()
    {
        System.out.println("Ini milik class Parent");
    }
}

class Child extends Parent
{
    public void Info()
    {
        System.out.println("Ini milik class Child");
    }
}
```

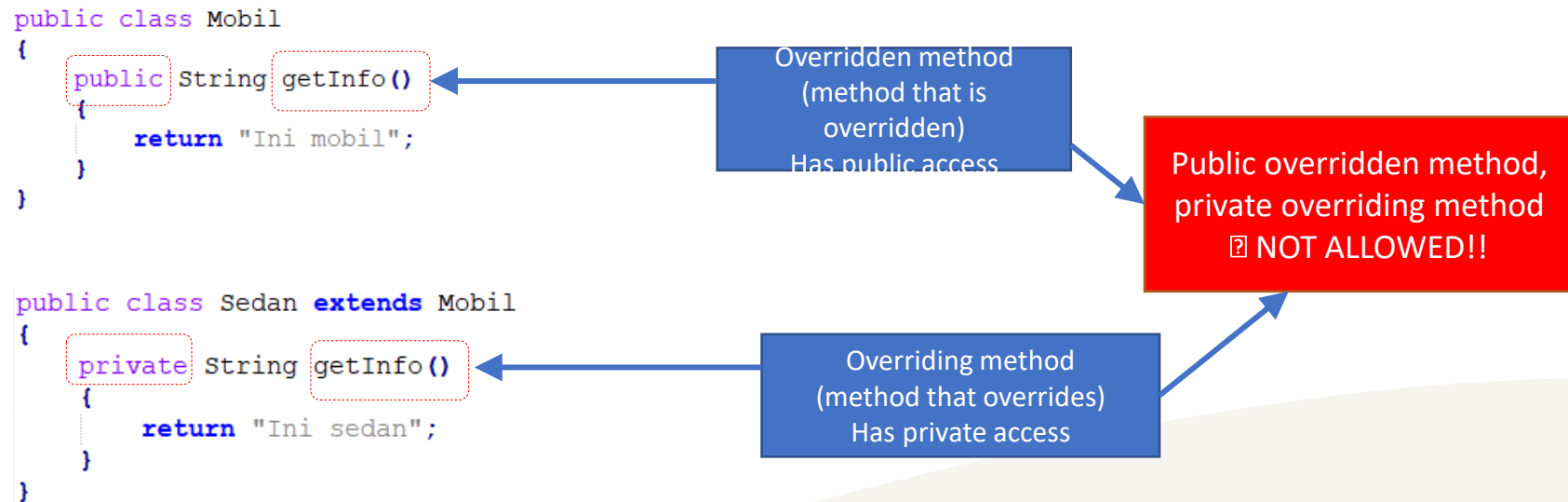


In the example above, the class Child is derived from the class Parent, and both the Parent and Child classes have the Info() method with exactly the same signature.

- This means the Info() method in the Parent class is overridden by the Child class

Overriding

- An overridden method must not have a wider access level than the method that overrides it. Example:



Example of Overriding

```
public class A
{
    public void tampilkanKeLayar()
    {
        System.out.println("Method milik class A dipanggil...");
    }
}

public class B extends A
{
    public void tampilkanKeLayar()
    {
        System.out.println("Method milik class B dipanggil...");
    }
}

public class TestOverriding
{
    public static void main(String[] args)
    {
        B objB = new B();
        objB.tampilkanKeLayar();
    }
}
```

```
D:\Java\Polimorfisme>java TestOverriding
Method milik class B dipanggil...
```

When to Use Overriding

- Overriding is used when there is a class that is a subclass of another class but wants to change the functionality of the inherited method without adding a new method.
- 

Real World Scenario of Using Overriding

- For example, there is an ElectricCar class that is a subclass of the Car class
- Inside the Car class, there is a method addSpeed()
- Then the addSpeed() method is also inherited by the ElectricCar class, but its implementation is different.
 - **addSpeed() in the Car class is implemented by increasing the supply of fuel and air to the engine**
 - **addSpeed() in the ElectricCar class is implemented by increasing the electrical voltage to the engine**

Real World Scenario of Using Overriding

```
public class Mobil
{
    public void tambahKecepatan()
    {
        // lakukan proses penambahan kecepatan
        // yaitu dgn menambah suplai bahan bakar dan udara ke mesin
    }
}

public class MobilListrik extends Mobil
{
    public void tambahKecepatan()
    {
        // lakukan proses penambahan kecepatan
        // yaitu dgn menambah tegangan listrik ke mesin
    }
}
```

- We can see in the example above that ElectricCar overrides the addSpeed() method from the Car class

Example of Overloading

- A gas station has an automatic filling feature. If what is filled is a luxury car, then fill it with pertamax. If it is an old car, then fill it with pertalite. Create a simple program using overloading, test with the following:

```
public class TestGasStation
{
    public static void main(String[] args)
    {
        MobilMewah alphard = new MobilMewah();
        MobilKuno carry = new MobilKuno();
        GasStation gs = new GasStation();

        gs.isiBahanBakar(carry);
        gs.isiBahanBakar(alphard);
    }
}
```

```
run:
Mobil kuno telah diisi dengan Pertalite!
Mobil mewah telah diisi dengan Pertamax!
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example of Overloading (continued)

```
public class MobilMewah
{
}

public class MobilKuno
{
}

public class GasStation
{
    public void isiBahanBakar(MobilMewah mw)
    {
        System.out.println("Mobil mewah telah diisi dengan Pertamina!");
    }

    public void isiBahanBakar(MobilKuno mk)
    {
        System.out.println("Mobil kuno telah diisi dengan Pertamina!");
    }
}
```

Example of Overriding

- An employee information system contains data about managers and supervisors. The supervisor is a derivative of the manager. The manager has name and salary attributes. The manager has a method to print status that prints all its attributes. All classes have a method to raise salary, which will increase the salary by: manager +1,000,000 and supervisor +1,500,000. Create the program and test it with the following:

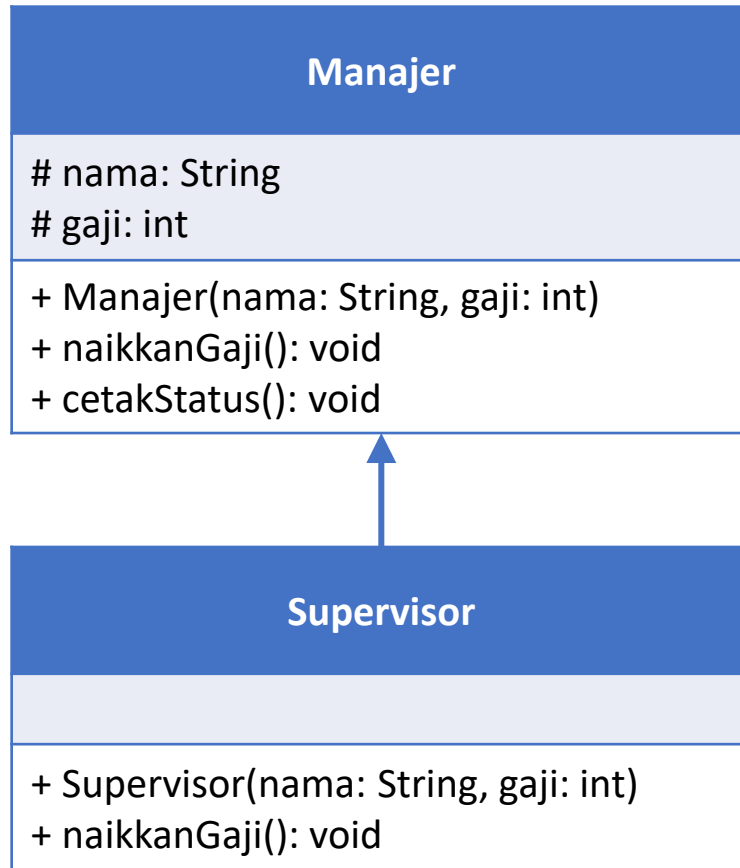
```
public class TestKepegawaian
{
    public static void main(String[] args)
    {
        Manajer man = new Manajer("Bill Gates", 5000000);
        Supervisor spv = new Supervisor("Susanto", 1000000);

        man.naikkanGaji();
        spv.naikkanGaji();

        man.cetakStatus();
        spv.cetakStatus();
    }
}
```

```
run:
Nama: Bill Gates
Gaji: 6000000
Nama: Susanto
Gaji: 2500000
BUILD SUCCESSFUL (total time: 0 seconds)
```

Example of Overriding (continued)



Example of Overriding (continued)

```
public class Manajer
{
    protected String nama;
    protected int gaji;

    public Manajer(String nama, int gaji)
    {
        this.nama = nama;
        this.gaji = gaji;
    }

    public void naikkanGaji()
    {
        gaji += 1000000;
    }

    public void cetakStatus()
    {
        System.out.println("Nama: " + nama);
        System.out.println("Gaji: " + gaji);
    }
}
```

```
public class Supervisor extends Manajer
{
    public Supervisor(String nama, int gaji)
    {
        super(nama, gaji);
    }

    public void naikkanGaji()
    {
        gaji += 1500000;
    }
}
```

Question?